INSTITUTO SUPERIOR TÉCNICO MILITAR

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

# BIBLIOTECA ISTM

## Relatório técnico do mini-projecto

Tema 2 - Sistema de Gestão de Biblioteca

---

Disciplina: Tópicos Complementares

Módulo: Engenharia de Software Avançada

Ano lectivo: 2026/2027

Grupo 2: Sistema de Gestão de Biblioteca (4 estudantes)

Membros: preencher nomes | Scrum Master: designar

Luanda | 4 de Novembro de 2026

# 1. Introdução e problema

O mini-projecto aplica práticas de Engenharia de Software Avançada a um sistema de gestão de biblioteca. O problema escolhido no Tema 2 é acompanhar o catálogo e a circulação: identificar livros e leitores, registar empréstimos e devoluções e manter a disponibilidade coerente com as operações realizadas.

Uma folha de registo ou uma aplicação sem validações pode permitir duplicados, emprestar um exemplar já requisitado ou fechar duas vezes a mesma devolução. A solução deve concentrar as regras num serviço de negócio, validar os dados antes de os guardar e demonstrar o comportamento com testes automatizados.

## Objectivo geral

Construir uma aplicação Java organizada em camadas que registe livros e leitores, controle empréstimos e devoluções e verifique as regras de negócio com JUnit 5, Maven e JaCoCo. Um workflow GitHub Actions foi preparado para executar a mesma validação de forma contínua quando o repositório remoto for criado.

## Âmbito entregue

A versão actual é Web e permite gerir exemplares, leitores, empréstimos, devoluções, alertas e suspensões. O acesso exige login com perfil Administrador ou Bibliotecário. O armazenamento é intencionalmente em memória nesta demonstração; os dados e contas criadas desaparecem ao reiniciar o processo.

## Critérios assumidos

Um código interno identifica cada exemplar físico; o ISBN bibliográfico poderá ser partilhado por várias cópias e ainda não é armazenado. Identificador e endereço de correio electrónico identificam um leitor. Um empréstimo é activo até a data de devolução ser gravada. O leitor tem de existir e o exemplar tem de estar disponível. A devolução não pode anteceder o empréstimo. Prazos, renovações, multas, reservas e limites de requisições não foram definidos sem validação institucional.

Este relatório descreve a implementação local verificada em 4 de Novembro de 2026. Identificação do grupo e papéis Scrum devem ser preenchidos com informação real antes da entrega académica.

# 2. Requisitos e Product Backlog

Os requisitos funcionais foram derivados do Tema 2 do enunciado. O backlog abaixo traduz os fluxos principais em histórias verificáveis; os critérios de aceitação foram exercitados na aplicação e nos testes.

| ID | História de utilizador | Critério de aceitação |
|---|---|---|
| US01 | Como bibliotecário, quero registar exemplares. | Código interno, título e autor obrigatórios; código duplicado rejeitado. |
| US02 | Como bibliotecário, quero consultar exemplares. | Pesquisa por código interno e lista mostram dados e disponibilidade. |
| US03 | Como bibliotecário, quero registar leitores. | Campos obrigatórios; e-mail validado e duplicados rejeitados. |
| US04 | Como bibliotecário, quero consultar leitores. | Pesquisa por ID e lista de leitores registados. |
| US05 | Como bibliotecário, quero emprestar livro disponível. | Livro e leitor existentes; empréstimo activo e estado actualizado. |
| US06 | Como bibliotecário, quero impedir empréstimos inválidos. | Livro indisponível e leitor desconhecido não criam empréstimo. |
| US07 | Como bibliotecário, quero registar devoluções. | Só empréstimo activo; data válida; disponibilidade restaurada. |
| US08 | Como bibliotecário, quero consultar a circulação. | Lista mostra código, livro, leitor e estado/data da devolução. |
| US09 | Como equipa, queremos evitar regressões. | Maven executa testes; JaCoCo falha abaixo de 70% na lógica. |
| US10 | Como equipa, queremos validar cada integração. | Workflow verifica pull requests e push para main; publica artefactos. |

## Requisitos não funcionais

A solução usa Java 17 e Maven, separa domínio, aplicação e acesso a dados, define interfaces para persistência substituível, executa testes sem intervenção manual e produz um relatório de cobertura. A arquitectura foi escolhida para favorecer manutenibilidade e testabilidade num problema de pequena escala.

A execução actual cumpre os fluxos de registo, consulta, disponibilidade, empréstimo e devolução. O backlog completo, incluindo os critérios e Definition of Done, está em docs/product-backlog.md.

# 3. Arquitectura e justificação

Foi adoptada uma arquitectura Web em camadas. Thymeleaf e Spring MVC recebem entradas do navegador e apresentam resultados. Spring Security protege as rotas e separa os perfis Administrador e Bibliotecário. LibraryService implementa os casos de uso; Book, Reader, Loan e Suspension representam o domínio. Interfaces de repositório isolam o armazenamento em memória.




| Estilo | Manutenibilidade | Testabilidade | Adequação ao projecto |
|---|---|---|---|
| Camadas | Responsabilidades separadas; interface e armazenamento podem mudar isoladamente. | Serviço recebe repositórios por interface; testes rápidos sem serviços externos. | Adequado ao âmbito reduzido e aos requisitos de separação. |
| Microserviços | Módulos independentes, mas exigem contratos e operação distribuída. | Testes de integração entre serviços aumentam a infraestrutura necessária. | Complexidade desproporcional para um catálogo e circulação locais. |
| Orientada a eventos | Desacopla consumidores assíncronos. | Exige testar publicação, entrega e ordenação de eventos. | Sem benefício claro num fluxo síncrono de empréstimo/devolução. |

A arquitectura em camadas satisfaz manutenibilidade e testabilidade com baixo custo operacional. O desenho evita dependência do serviço em relação às classes concretas de armazenamento. Quando a instituição escolher uma base de dados, basta acrescentar adaptadores para os contratos existentes e revalidar as regras através da suite.

# 4. Implementação, padrões e refactoring

A aplicação segue a estrutura convencional de Maven. domain contém Book, Reader, Loan e Suspension; service implementa os casos de uso; repository define os contratos e os adaptadores em memória. LibraryWebController liga páginas Thymeleaf ao serviço; o pacote security trata login e perfis. LibraryFactory compõe as dependências.

A marca actual é um logótipo gráfico próprio do projecto, com livro aberto e iniciais ISTM. Foi adoptada uma paleta académica azul-marinho e dourada; ícones SVG locais evitam dependências externas. O activo não é apresentado como brasão oficial da instituição.

A interface Web usa Spring MVC e Thymeleaf para gerar páginas HTML no servidor. As páginas e estilos ficam separados em templates e CSS, permitindo mudar navegação, campos e aspecto visual sem alterar as regras de negócio. O servidor local pode ser aberto em http://localhost:8080.

## Padrão Repository

Os contratos de repositório abstraem operações de gravação e consulta. Os adaptadores em memória implementam esses contratos com mapas. LibraryService depende das interfaces, o que permite substituir a estratégia de persistência sem alterar os casos de uso. Esta escolha cumpre o padrão Repository pedido e cria uma fronteira testável para o acesso aos dados.

## Padrão Factory

LibraryFactory.createInMemoryLibrary() instancia os adaptadores e injecta-os no serviço. A composição fica centralizada e oferece um ponto de substituição quando for escolhida uma base de dados.

## Ciclo de refactoring

Problema detectado: a inicialização da interface criava directamente cada repositório, ligando apresentação a infraestrutura. Alteração: a composição foi extraída para LibraryFactory. Métrica simples: referências da classe de aplicação às implementações concretas passaram de três para zero. A prova de regressão é a suite de serviço, Web e integração; o teste de integração completa o ciclo de empréstimo e devolução.

## Validação adicional

A operação de empréstimo e devolução é sincronizada na instância do serviço para evitar duas operações simultâneas na mesma instância. Isto não substitui uma transacção numa base de dados partilhada; a solução actual é local e em memória.

# 5. Estratégia e casos de teste

JUnit Jupiter executa os testes pelo ciclo Maven. A suite combina testes unitários das entidades e regras de LibraryService com testes de integração que usam repositórios reais em memória e testes MVC das páginas, operações, autenticação e privilégios.

| Grupo de cenários | Casos cobertos | Resultado esperado |
|---|---|---|
| Entidades | Campos nulos/vazios; e-mail inválido; igualdade por identificador; estados repetidos. | Dados inválidos rejeitados; estados e igualdade consistentes. |
| Catálogo e leitores | Registo, listagem, códigos internos/ID/e-mail duplicados e consultas inexistentes. | Registos válidos guardados; duplicados e ausentes tratados. |
| Empréstimo | Livro disponível/indisponível; leitor desconhecido; datas ausentes; empréstimo activo duplicado. | Só operações permitidas criam empréstimo e alteram disponibilidade. |
| Devolução | Devolução válida, data anterior, empréstimo inexistente ou já fechado. | Estado fecha uma vez e disponibilidade regressa; erros não alteram estado. |
| Integração | Factory, serviço e três repositórios num ciclo completo. | Dados mantêm coerência entre serviço e adaptadores. |
| Consola | Opções do menu, registos, consulta, empréstimo, devolução e mensagens de erro. | Entrada textual invoca operações e apresenta resultados legíveis. |

A execução final incluiu 30 testes automatizados: cinco de domínio, um de integração, dois de segurança, catorze do serviço e oito Web. Os testes incluem prazos, atraso, suspensão, bloqueio de novo empréstimo, páginas, pesquisa, CSRF e autorização por perfil.

# 6. Resultados de testes e cobertura

A validação foi executada localmente em 4 de Novembro de 2026 com `mvn verify`, repositório Maven no projecto, Java 25 e compilação configurada para Java 17. A execução terminou com BUILD SUCCESS. Os 30 testes passaram, sem falhas, erros ou ignorados.

**JaCoCo 0.8.16 analisou 18 classes de produção. A cobertura foi 280/299 linhas (93,6%) e 107/138 ramos (77,5%). O pacote de negócio superou o limiar configurado de 70% de linhas.**

| Área | Linhas | Ramos | Leitura |
|---|---|---|---|
| Total do projecto | 280/299 - 93,6% | 107/138 - 77,5% | Limiar de 70% nas regras de negócio superado. |
| LibraryService | 45/47 - 95,7% | 30/36 - 83,3% | Regras, prazos e suspensões exercitados. |
| Domínio | 66/69 - 95,7% | 36/38 - 94,7% | Validações e transições de estado cobertas. |
| Interface Web | 86/95 - 90,5% | 13/22 - 59,1% | Páginas, fluxos e autorização testados com MVC. |
| Segurança | 54/57 - 94,7% | 22/36 - 61,1% | Login, perfis e gestão de contas exercitados. |
| Repositórios em memória | 24/24 - 100% | 6/6 - 100% | Gravação, consulta e adaptadores cobertos. |

A cobertura dos pacotes Web e security ainda pode crescer, sobretudo em ramificações de erro e combinações de perfis. JaCoCo é uma métrica diagnóstica, não prova de correcção; os testes verificam regras de circulação, restrições e alguns fluxos de autenticação.

O limiar automatizado no POM é 70% de linhas no pacote de negócio. O relatório local ficou acima do limiar em todo o projecto; a cobertura de ramos é apresentada como diagnóstico adicional, não como garantia isolada de correcção.

# 7. Pipeline CI/CD

O workflow .github/workflows/ci.yml está preparado para executar em push para main e em pull requests dirigidas a main. Usa runner Ubuntu, configura Temurin Java 17 e executa `mvn --batch-mode --no-transfer-progress clean verify`. O passo de verificação inclui compilação, testes JUnit, relatório JaCoCo e bloqueio abaixo de 70% de linhas nas classes de negócio.

## Artefactos definidos

O workflow disponibiliza o JAR e a pasta do relatório HTML JaCoCo como artefactos da execução. O upload usa `if: always()` para preservar os relatórios quando existe uma falha, se os ficheiros tiverem sido produzidos. A cache Maven é configurada através de setup-java.

## Evidência local

A mesma etapa `mvn clean verify` foi executada no ambiente local. Evidências disponíveis localmente: relatório HTML JaCoCo em target/site/jacoco/index.html, ficheiro XML em target/site/jacoco/jacoco.xml e JAR em target/biblioteca-istm-1.0.0-SNAPSHOT.jar.

## Estado da integração contínua

O workflow está escrito e pronto para integração, mas ainda não foi executado no GitHub: o utilizador definiu que as melhorias e experiências locais antecedem a criação/envio do repositório. Portanto, não há execução hospedada a capturar nem se afirma que o pipeline remoto já passou. Ao preparar o repositório, a primeira execução deverá confirmar o JDK, dependências, permissões de artefactos e resultado do limiar JaCoCo.

Uma falha de compilação ou de teste termina `verify` com resultado não zero e deixa a execução vermelha. O passo de upload, por estar configurado para sempre, tenta conservar o JAR e o relatório para diagnóstico.

# 8. Processo de desenvolvimento

O enunciado define um sprint único e pede papéis, quadro, daily Scrum, Definition of Done, revisão por Pull Request e retrospectiva. Esta cópia local já documenta histórias, critérios e Definition of Done em docs/product-backlog.md. A identidade dos membros e a atribuição real dos papéis não foram fornecidas e permanecem por preencher.

| Papel / actividade | Estado factual |
|---|---|
| Product Owner | Docente, conforme o enunciado. |
| Scrum Master e Developers | A designar/preencher pelos membros reais do grupo. |
| Quadro e daily Scrum | Não iniciados localmente; registar actualizações reais quando a equipa trabalhar no sprint. |
| Definition of Done | Compilação, testes, pipeline verde e revisão por outro membro, conforme o enunciado. |

## Retrospectiva técnica local

Nesta fase local, correu bem separar regras, domínio e armazenamento desde o primeiro protótipo e verificar as operações com testes pequenos. A primeira versão ligava a inicialização da interface aos repositórios concretos; a Factory removeu essa dependência e o teste de integração confirmou que o ciclo de empréstimo continuou a funcionar. A análise de cobertura também revelou ramos não exercitados no domínio e no menu. Foram acrescentados cenários de dados inválidos, transições repetidas, consultas e interacção de consola até a suite cobrir todas as linhas e ramos. A revisão do modelo identificou ainda que ISBN pode ser partilhado por várias cópias, pelo que o exemplar passou a usar código interno único. Esta iteração mostrou que cobertura alta só é útil quando os testes verificam regras e estados relevantes. No próximo ciclo, o grupo deve confirmar com a biblioteca os prazos, renovações, exemplares e perfis de operador antes de escolher persistência durável. Esta retrospectiva descreve o trabalho local e não substitui o registo real do sprint da equipa.

## Métricas exigidas

| Métrica | Resultado nesta fase |
|---|---|
| Tempo médio de fecho de Pull Requests | Não disponível: nenhuma Pull Request foi criada. |
| Execuções do pipeline remoto | 0: workflow ainda não executado no GitHub. |
| Percentagem de pipelines falhados | Não calculável sem execuções. |

Não se atribuem valores estimados a métricas que dependem do histórico remoto. Depois da integração no GitHub, os resultados devem ser extraídos do repositório e acrescentados ao relatório.

# 9. Conclusão, limitações e referências

O projecto implementa o núcleo do Tema 2 através de uma aplicação exclusivamente Web, com login, perfis, catálogo, circulação, alertas e suspensão em caso de atraso. A execução local passou 30 testes e obteve 93,6% de cobertura de linhas total; as regras de negócio superaram o limiar JaCoCo de 70%. O workflow CI está preparado para validação futura no GitHub.

## Limitações e evolução

A persistência é temporária; utilizadores e empréstimos desaparecem ao reiniciar. Os valores de 14 dias para empréstimo e 7 dias para suspensão são demonstrativos e configuráveis. A instituição deve confirmar a política de empréstimo e sanções, definir reservas, notificações e login de leitores, escolher armazenamento durável e backup. As experiências locais precedem a publicação no GitHub.

## Referências

Apache Software Foundation. (n.d.). Maven getting started guide. https://maven.apache.org/guides/getting-started/index.html

GitHub. (n.d.). Building and testing Java with Maven. https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-maven

Instituto Superior Técnico Militar. (2026). Enunciado do mini-projecto em grupo: Tópicos Complementares, módulo Engenharia de Software Avançada. Documento fornecido na disciplina.

JUnit Team. (2024). JUnit 5.11.4 user guide. https://docs.junit.org/5.11.4/user-guide/junit-user-guide-5.11.4.pdf

JaCoCo Team. (2026). JaCoCo 0.8.16: Maven plug-in. https://www.jacoco.org/jacoco/trunk/doc/maven.html

Oracle. (n.d.). Java Platform, Standard Edition 17 documentation. https://docs.oracle.com/en/java/javase/17/

Anexo digital: o código, o workflow, os testes e o relatório JaCoCo acompanham esta versão local do projecto.